

Real-time Rendering for Virtual Humans: Animating Blendshapes

Mentor: Kalle Bladin
Jonathan Kaneshiro

USC Institute for
Creative Technologies



About

- Fourth Year Computer Science undergraduate student
- Computer Graphics
- Interests: Animation, Visual effects, Simulation, Augmented / Virtual Reality, Rendering, UI / UX, Web development
- Creative technologist / Media artist
- Soccer, concerts, museums

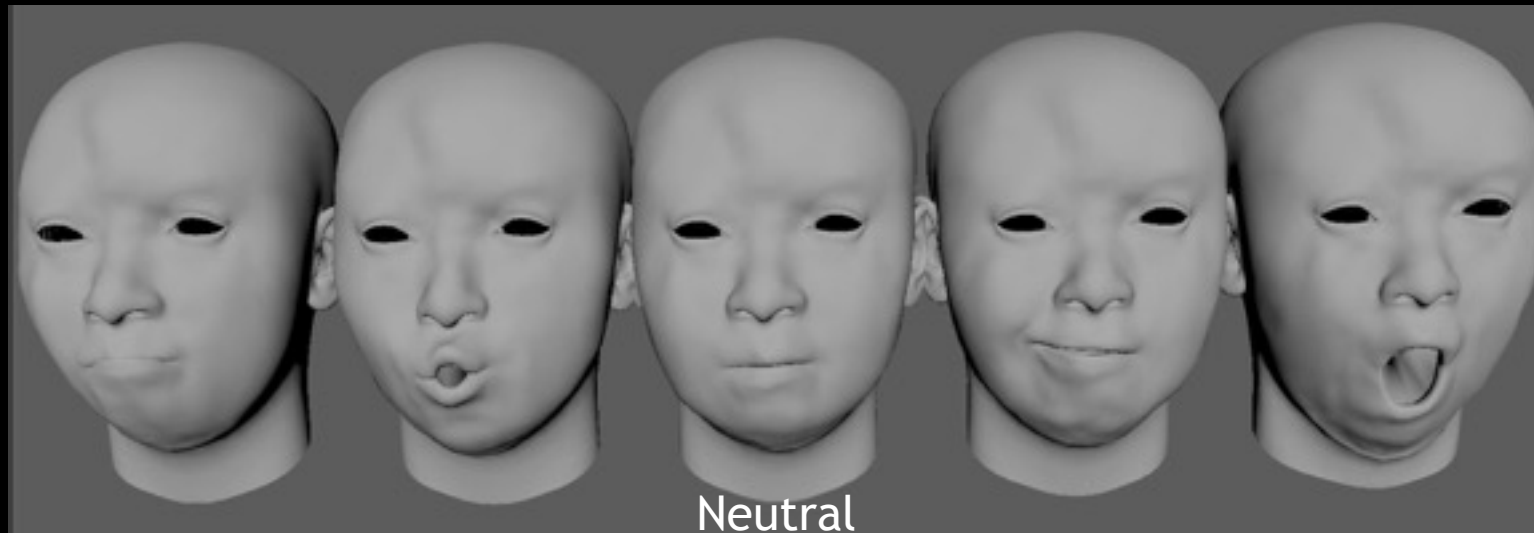


Motivation

- Real-time Rendering for Virtual Humans:
 - **photo realistic virtual humans** using light stage data
 - quality control of processing pipeline data for **client delivery**
 - data for **deep learning** purposes
 - renderings for **publications**
- Grendel (Unity-based Rendering Software) doesn't support animation of blendshapes
- Technically it does, but not in our way
- That's where I come in!

Blendshapes

- A blendshape model generates a facial pose as a **linear combination of a number of facial expressions**, the blendshape “targets”.
- Each target can be a **complete** facial expression, or a “**delta**” expression such as raising one of the eyebrows [Lewis et al.]



Facial Rig

- Captured using Kinects
- Faceshift
- Provided with 48 facial expressions (.obj)
- Merged meshes into a blendshape in Maya (.fbx)
- Output: animation text file



Blendshape Animation

Parsing the Animation File

FS	5	I	0	1	P	-0.0240764	0.0438273	0.00502243	0.998736	-57.6353	-13.0079	-883.676	C	48	0.194776	0	0	1	0.817541	0.826361	1	0	0	0.107663
FS	5	I	33.369	1	P	-0.0233441	0.0422324	0.0169132	0.998692	-55.0514	-12.8032	-884.102	C	48	0.227221	0	0	1	0.814102	0.822623	1	0	0	0.17
FS	5	I	66.739	1	P	-0.02821	0.0412347	0.00960973	0.998705	-55.9502	-13.8738	-882.972	C	48	0.196148	0	0	1	0.828365	0.837079	1	0	0	0.203
FS	5	I	100.108	1	P	-0.0296246	0.0393952	0.0105909	0.998728	-55.2407	-14.4458	-882.375	C	48	0.170659	0	0	1	0.835486	0.844163	1	0	0	0.2
FS	5	I	133.477	1	P	-0.0281403	0.039409	0.0123048	0.998751	-55.093	-14.3066	-882.515	C	48	0.154732	0	0	1	0.838352	0.846832	1	0	0	0.222
FS	5	I	166.846	1	P	-0.0301397	0.039375	0.00882675	0.998719	-54.8316	-15.031	-882.146	C	48	0.127664	0	0	1	0.856438	0.865431	1	0	0	0.18
FS	5	I	200.216	1	P	-0.0322263	0.0385882	0.00816679	0.998687	-55.51	-14.882	-881.746	C	48	0.146975	0	0	0.999093	0.863034	0.872196	1	0		
FS	5	I	233.585	1	P	-0.0295189	0.0390352	0.00874957	0.998754	-55.5157	-14.3274	-882.147	C	48	0.169902	0	0	0.981123	0.846269	0.855227	1			
FS	5	I	266.954	1	P	-0.0298453	0.0398429	0.00900391	0.998711	-55.3181	-14.2474	-882.379	C	48	0.213396	0	0	0.918239	0.833854	0.84285	1			
FS	5	I	300.323	1	P	-0.0277613	0.0405433	0.0107758	0.998718	-55.3735	-13.3122	-882.878	C	48	0.273758	0	0	0.847371	0.821838	0.830473	1			
FS	5	I	333.693	1	P	-0.0293919	0.0392279	0.0091925	0.998745	-55.5065	-13.4945	-882.374	C	48	0.262082	0	0	0.780743	0.833783	0.842522	1			

Each line contains

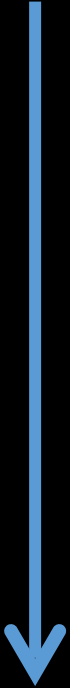
FS: Frame

P: Head pose (quaternions & translations)

C: Coefficients of each 48 facial movements

Storing Data

Frames



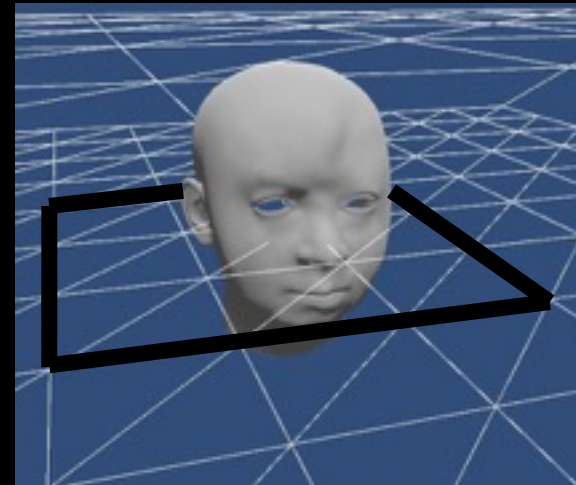
(a, b, c, d)	(x, y, z)	
Quaternions	Translations	Coefficients: 48 facial expressions
Quaternions	Translations	Coefficients: 48 facial expressions
Quaternions	Translations	Coefficients: 48 facial expressions
Quaternions	Translations	Coefficients: 48 facial expressions
Quaternions	Translations	Coefficients: 48 facial expressions
Quaternions	Translations	Coefficients: 48 facial expressions
Quaternions	Translations	Coefficients: 48 facial expressions
...		
Quaternions	Translations	Coefficients: 48 facial expressions

2D Array

Positioning the Facial Rig

- Min-Max Normalization
- **[a,b]** -> localized range
- **x** -> each element in the array
- **min / max x** -> each component of rotation and position arrays
- **x'''** -> localized element (referenced to 2D array)

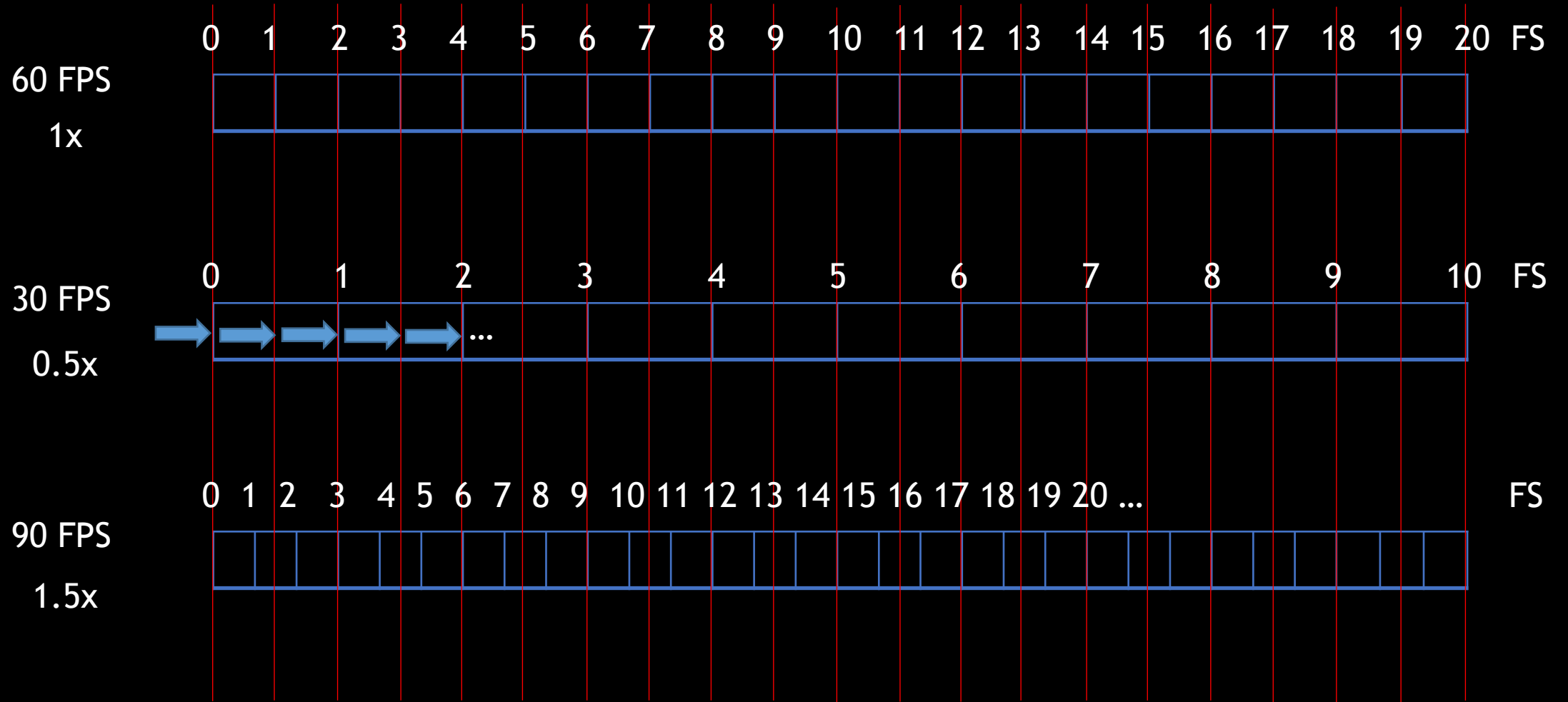
$$x''' = (b - a) \frac{x - \min x}{\max x - \min x} + a$$



Calculating the FPS Ratio

- Why?
- Unity's Update() is called each frame
- Can't change frame rate
- Get Unity's current frame rate (60 FPS)
- Want a desired frame rate (30 FPS)
- $\text{FPSratio} = \text{Desired FPS} / \text{Current FPS} = (30/60) = (1/2)$
- $\text{RatioIncrementor} = 1/2$

Animating



Linear Interpolation

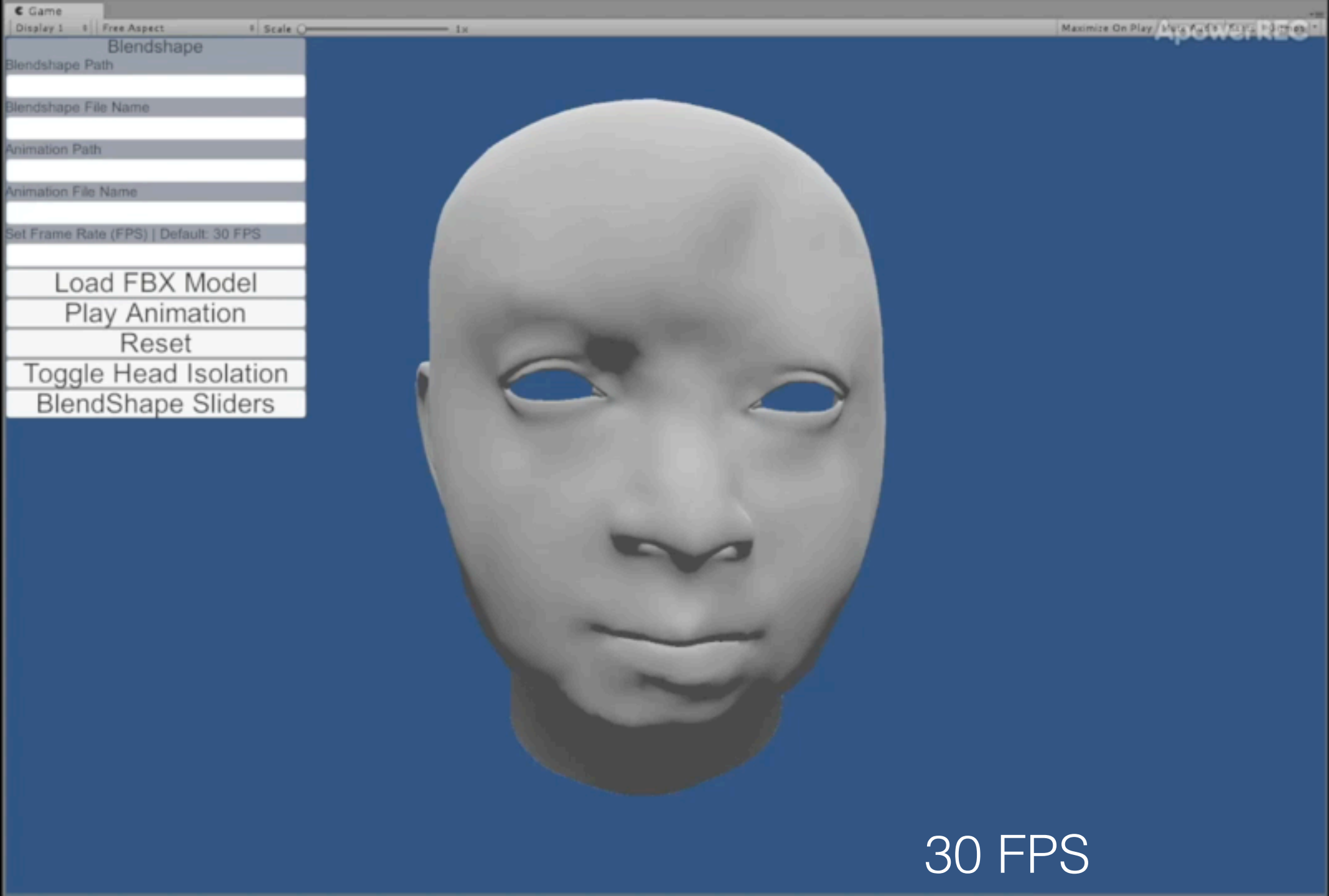
```
float lerp(float v0, float v1, float t) {  
    return (1 - t) * v0 + t * v1;  
}
```

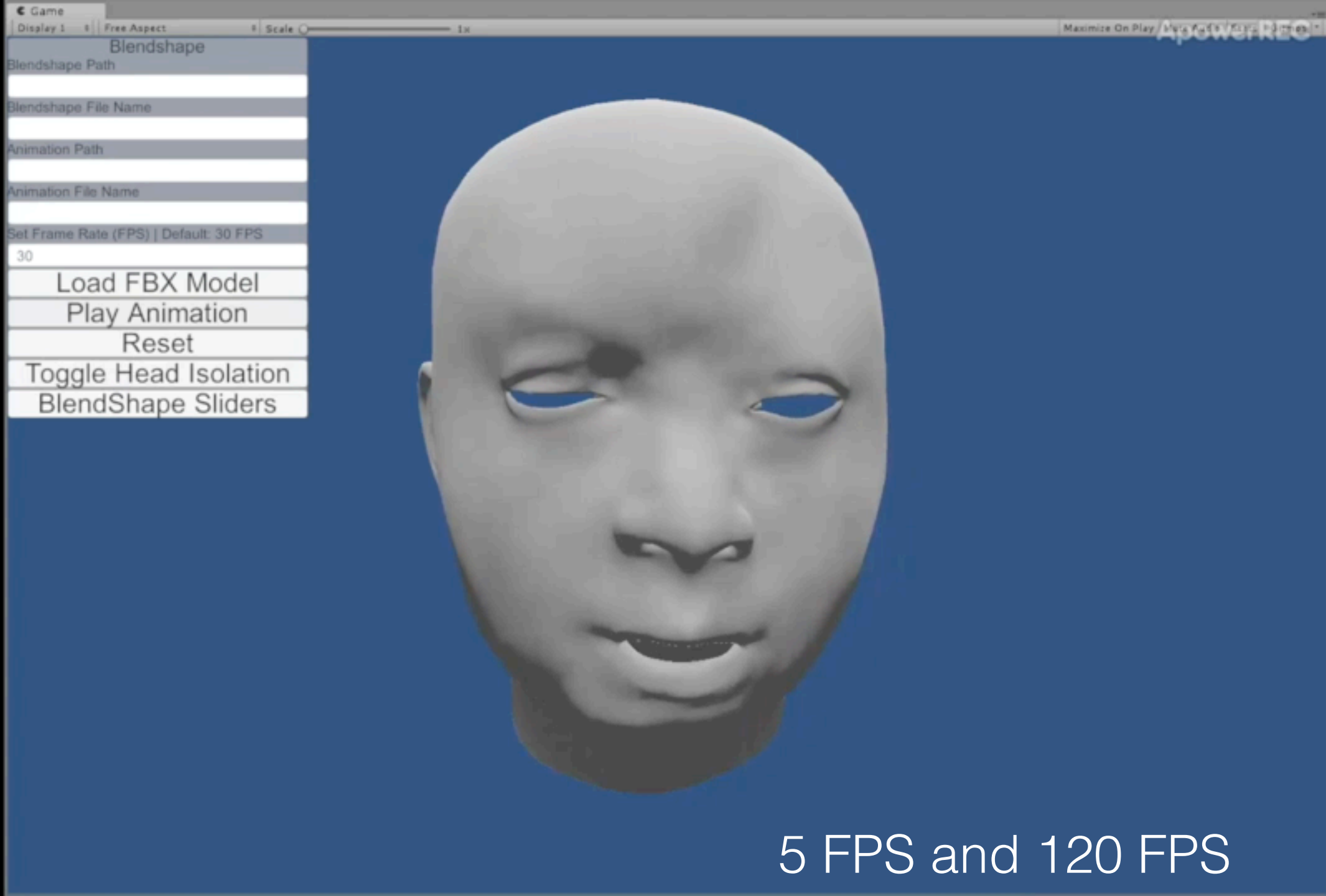
V0 -> floor(ratioIncr)

V1 -> ceil(ratioIncr)

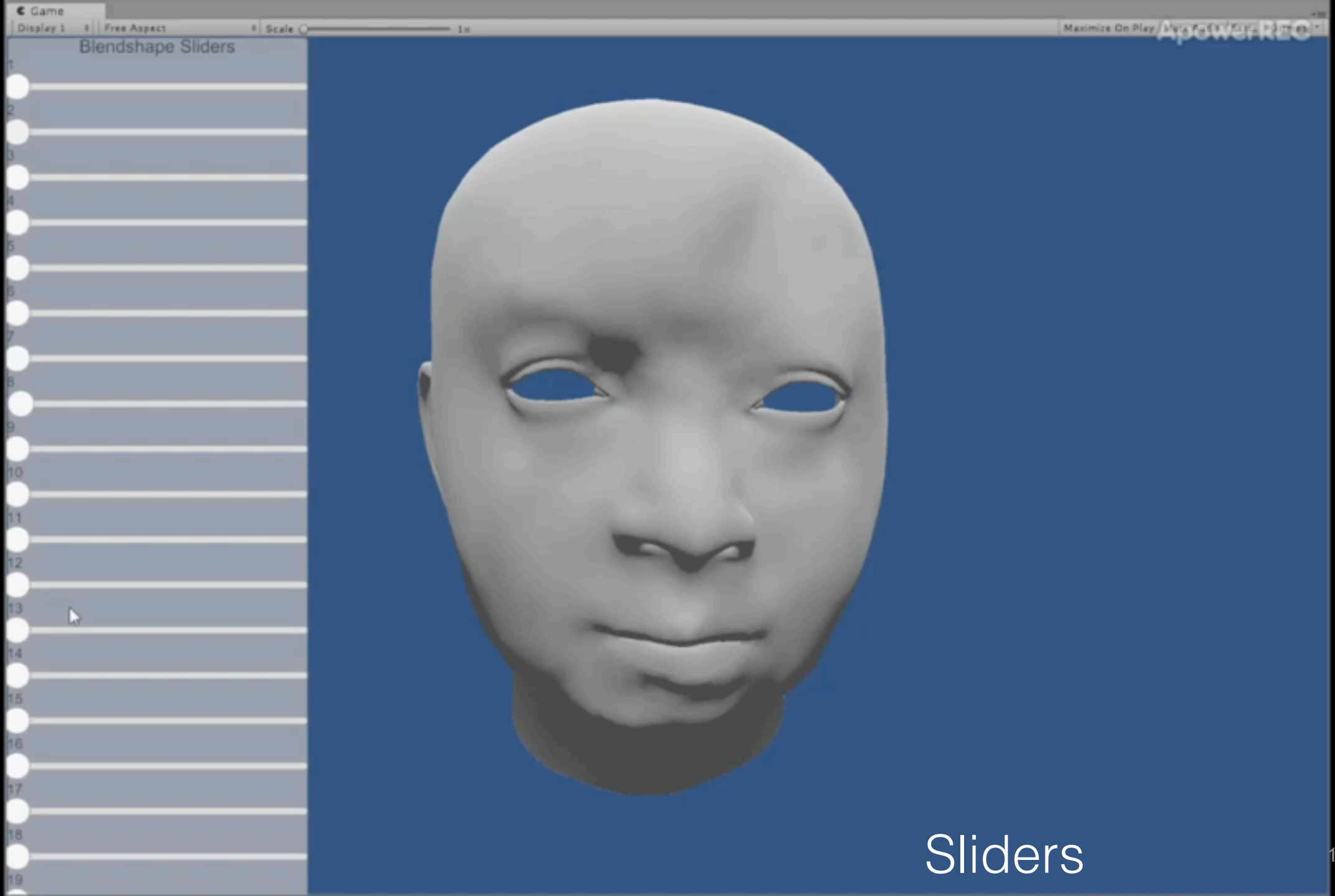
t -> ratioIncr

Demos





5 FPS and 120 FPS



Sliders

Application for the Light Stage

- Unified display for animating facial rigs and showing various expressions
- Animation text file for real-time rendering
- As of now, sliders work with any imported Blendshapes
- But, animation works with certain rigs

Future Work

- Creating a blendshape rig within Grendel
 - Folder containing **multiple meshes** (.OBJs) merging into a blendshape rig (.FBX)
 - Storing and tracking **vertices, tangents, and normals** of meshes
- **Single** animation text file used across various blendshape rigs
- Cluster involvement -> tracking **landmarks** for real time animation (emotional expressions)
- Adjust for character **skinning**, if possible

Thank you for having me!